

Python Numbers & Mathematical Operators: A Reference

Core Calculation Engines, Operator Mechanics & Syntax Workspaces

Python handles numerical calculations dynamically using distinct numeric types. This reference covers core numerical types, standard arithmetic operations, floor division variations, modulo tracking, and the rules governing calculation precedence in analytical workflows.

1. Core Numeric Types

Data Type	Instruction & Core Mechanics	Step-by-Step Code & Output
int	Integers: Whole numbers positive, negative, or zero, without decimals. Python manages integer sizing dynamically under the hood.	<pre>>>> x = 10 >>> type(x) <class 'int'></pre>
float	Floating-Point Numbers: Numbers containing a decimal point or written using scientific E-notation for fractional precision.	<pre>>>> y = 10.5 >>> type(y) <class 'float'></pre>

2. Standard Arithmetic Operators

Operator	Instruction & Core Mechanics	Step-by-Step Code & Output
+ (Addition)	Adds numeric values together. Adding an int to a float automatically converts the result to a float.	<pre>>>> 5 + 3.0 8.0</pre>
- (Subtraction)	Subtracts the right operand from the left operand.	<pre>>>> 10 - 4 6</pre>
* (Multiplication)	Multiplies numeric values together.	<pre>>>> 3 * 4 12</pre>
/ (Division)	Standard division. Crucially, standard division in Python 3 always returns a float type, even if the division evaluates cleanly.	<pre>>>> 10 / 2 5.0</pre>

3. Advanced Division, Remainder, & Powers

Operator	Instruction & Core Mechanics	Step-by-Step Code & Output
// (Floor Division)	Divides numbers and floors the result to the nearest full whole integer downward, stripping away any fractional remainder.	<pre>>>> 11 // 3 3 >>> -11 // 3 -4</pre>

Operator	Instruction & Core Mechanics	Step-by-Step Code & Output
% (Modulo)	Returns the remainder left over after executing integer division loops. Highly useful for tracking even or odd indexing sequences.	<pre>>>> 11 % 3 2 >>> 10 % 2 0</pre>
** (Exponentiation)	Raises the left operand to the power of the right operand. Do not mistake this for the standard carat (^) operator.	<pre>>>> 2 ** 3 8</pre>

4. Operator Precedence (PEMDAS / BODMAS)

Python calculates multi-operator chains strictly following mathematical order of operations rule-sets.

Precedence Tier	Instruction & Core Mechanics	Step-by-Step Code & Output
1. Parentheses	Expressions inside brackets () force immediate execution priority, shifting execution chains.	<pre>>>> (2 + 3) * 4 20</pre>
2. Exponents	Power functions are processed immediately after parentheses evaluations.	<pre>>>> 2 * 3 ** 2 18</pre>
3. Mult / Div	Multiplication, division, modulo, and floor division share equal hierarchy and evaluate from left to right.	<pre>>>> 10 - 4 / 2 8.0</pre>
4. Add / Sub	Addition and subtraction hold the lowest priority tier and execute last.	<pre>>>> 10 - 4 + 2 8</pre>